

Relazione Progetto

☒ Completo



Introduzione al Progetto "Connections"

Il presente progetto consiste nello sviluppo di un sistema distribuito di gioco ispirato a "**Connections**"

(https://en.wikipedia.org/wiki/The_New_York_Times_Connections). Il sistema è basato su un'architettura **client-server centralizzata** progettata per gestire partite globali e simultanee tra molteplici utenti.

Obiettivo del gioco

L'obiettivo del giocatore consiste nell'aggregare le 16 parole fornite in quattro gruppi tematici ciascuno composto da quattro parole, sulla base di caratteristiche comuni. Il giocatore vince se riesce a individuare correttamente tre gruppi (il quarto è dedotto per esclusione) prima dello scadere del tempo prefissato o del raggiungimento del limite massimo di 4 errori.

Architettura di riferimento

Il sistema è suddiviso in due componenti principali:

1. **Server:** Agisce come coordinatore globale, gestendo il database dei puzzle, la persistenza dei profili utente e la sincronizzazione temporale delle partite. Utilizza un approccio **multithreaded** con thread pooling per gestire uno svariato numero.
2. **Client:** Fornisce l'interfaccia a linea di comando per l'interazione del giocatore. Gestisce la comunicazione con il server tramite connessioni **TCP persistenti (NIO)** e rimane in ascolto per notifiche asincrone (es. fine partita o notifiche vittoria/sconfitta) tramite il protocollo **UDP**.

Gestione delle Partite

Esiste un'unica partita attiva per tutti i giocatori presenti e questa partita cambia ogni 10 minuti: qualora il giocatore finisse la partita prima di questo tempo

dovrebbe aspettare lo scadere del tempo rimanente alla partita per partecipare alla partita successiva

Scelte progettuali

- **Thread Pool:** Invece di allocare un thread per ogni client, si è scelto di utilizzare un `ExecutorService` con un **Fixed Thread Pool (limita consumo massimo di risorse del server)**
- **GameTimer:** Al fine di dividere le partite in periodi temporali È stato implementato un `GameTimer` **centralizzato** basato su uno `ScheduledExecutorService`.
- **Salvataggio su file:** I file `users.json` e `matches.json` vengono aggiornati ogni volta che un match viene finalizzato o un utente si registra.
- **Comunicazione Udp:** il client passa il proprio indirizzo IP e la propria porta udp durante il login di un certo utente e salva le informazioni in una Map condivisa. Quando l'utente effettuerà il logout le informazioni verranno cancellate dalla mappa. Inoltre, il client attende le comunicazioni udp su un thread attivo in background
- **Serializzazione/deserializzazione:** vengono usate API della libreria GSON. Per deserializzare il file contenente tutte le 911 partite viene usata la streamingAPI, per serializzare/deserializzare messaggi tra client/server o informazioni su utenti e partite viene usata la data binding API

Schema generale dei thread

Lato Server:

- **Main Acceptor Thread:** Gestisce il `ServerSocket` primario. Quando un client si connette, delega immediatamente la gestione della comunicazione a un thread del pool.
- **Thread Pool (ExecutorService):** Utilizza un pool di 10 thread fissi per gestire le istanze di `ClientHandler`. Ogni thread gestisce la sessione TCP persistente di un singolo utente (richieste di login, invio proposte, richieste statistiche).

- **Game Scheduler Thread:** Un thread dedicato (basato su `ScheduledExecutorService`) che gestisce il timer della partita globale. Allo scadere del tempo, calcola i risultati finali e invia le notifiche UDP.

Lato Client

- **Main Interaction Thread:** Gestisce l'input da terminale e l'invio delle richieste al server.
- **UDP Listening Thread:** Un thread attivo in background, responsabile della ricezione asincrona di notifiche dal server (es. "Partita Conclusa") senza interrompere l'input dell'utente.

Definizione delle strutture dati

Lato Server:

Oggetti di Dominio

- **Classe `User`** : Rappresenta l'entità utente. Contiene le credenziali e le statistiche storiche aggregate come `totalScore` , `currentStreak` , `bestStreak` e i contatori per l'istogramma degli errori (`zeroMistakeGames` , `oneMistakeGames` , ecc.).
- **Classe `ServerGameState`** : Gestisce lo stato globale del server. Include la `gameHistory` , organizzata come una **mappa nidificata** `Map<Integer, Map<String, UserGameState>>` . Questa struttura è condivisa tra i thread ed è implementata da una `concurrenthashmap`. Permette di risalire velocemente allo stato di un giocatore (`UserGameState`) partendo dall'ID della partita. Lo stato del server viene letto/scritto da/su un file Json per garantire persistenza delle partite.
- **Classe `UserGameState`** : Memorizza i progressi di un singolo utente in una specifica partita: parole già indovinate, gruppi completati, numero di errori commessi e altre statistiche utili.
- **Classe `Game`** : Modella il puzzle caricato dal file JSON. Contiene una lista di `WordGroup` e l'id della partita corrente.
- **Classe `WordGroup`** : ogni oggetto `WordGroup` è composto da un tema e dalla lista di 4 parole associate a quel tema.

- Classe `PlayerRank` : mantiene solo alcune statistiche dello User, in modo da non far arrivare dati più sensibili come password al client. Usata per la classifica.

Strutture di Gestione

- In `userManager` : Utilizzo di una `ConcurrentHashMap<String, User>` . Permette al server di trovare istantaneamente un utente tramite il suo username in modo thread-safe. Questa mappa è condivisa tra i vari thread del threadpool e viene scritta/caricata in/da un file in formato Json per garantire la persistenza degli utenti anche a seguito di fault di un server.
- In `GameManager` : Una `List<Game>` `allGames` per contenere tutti i 911 puzzle disponibili e una variabile `currentGame` per il `Game` attualmente attivo per tutti i giocatori. La lista è condivisa ma read only, quando modifico la variabile `currentGame` lo faccio all'interno di un monitor

Lato client:

- `ByteBuffer` : Utilizzato nei metodi `sendJson` e `receiveJson` . Funge da memoria intermedia tra la rete e l'applicazione.
- `StringBuilder` : Utilizzato in `receiveJson` . Accumula i "chunk" (pezzi) di messaggi che arrivano dal canale TCP finché non viene trovato il carattere terminatore `\n` , garantendo che il JSON sia completo prima del parsing.

Condivise:

- Classe `GameMessage` : Trasforma i comandi mandati dall'utente in messaggi che verranno serializzati e inviati al server e viceversa.

Gestione concorrenza

Collezioni Thread-Safe

La struttura portante della persistenza in memoria si basa su collezioni della libreria `java.util.concurrent` :

- **ConcurrentHashMap**: Utilizzata in `userManager` e `ServerGameState` per gestire l'anagrafica utenti e la cronologia delle partite.

Monitor

In scenari dove l'operazione richiede atomicità su una sequenza di "Lettura-Modifica-Scrittura", sono stati utilizzati metodi sincronizzati:

- **Metodo `finalizeMatch`**: Questo metodo è marcato come `synchronized`. Impedisce che il thread del timer (allo scadere del tempo) e il thread del client (all'invio dell'ultima parola) tentino di chiudere la stessa partita simultaneamente, evitando l'assegnazione doppia di punteggi o l'incremento errato delle streak.
- **Metodo `getPlayerState`**: Sincronizzato per garantire che la creazione o il recupero dello stato di un giocatore avvenga in modo atomico, prevenendo la duplicazione degli oggetti `UserGameState` per la stessa partita.
- **`rotateGame` e `startNewGame`**: Questi metodi cambiano il riferimento a `currentGame`.

Sincronizzazione dell'Input/Output (File JSON)

- **Accesso Esclusivo ai File**: La parola chiave `synchronized` sui metodi di salvataggio assicura che un solo thread alla volta possa scrivere sui file `users.json` o `matches.json`. Questo previene eccezioni di I/O e garantisce che il file JSON rimanga ben formato e leggibile per i riavvii futuri del server.

Manuale di istruzioni

- **Librerie Esterne**: È richiesta la libreria **GSON** per la gestione dei messaggi in formato JSON e per la persistenza dei dati.
- **Dipendenze Dati**: Il server richiede la presenza della cartella `data/` contenente il file `games.json` con le partite.

Esecuzione del Sistema

Avvio del Server: Il sistema richiede l'avvio del server prima di poter connettere i client. All'avvio, il server caricherà i file `users.json` e `matches.json` per ripristinare lo stato precedente, se esistenti.

Avvio del Client: Il client tenterà la connessione TCP sulla porta predefinita e rimarrà in ascolto per notifiche UDP asincrone.

Una volta avviato il client, l'utente può interagire tramite l'interfaccia a riga di comando. Per avere la lista di comandi usabili l'utente può digitare il comando "help".

